

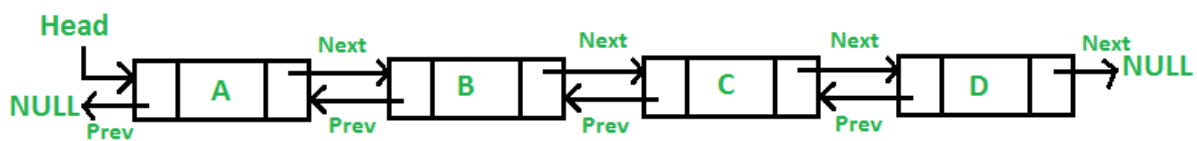
Programmieren II Probeklausur WS 22/23

Die Klausur besteht aus **drei unabhängigen Aufgaben**. Legen Sie für jede Aufgabe ein **eigenes Verzeichnis im Ordner Klausur** (auf dem Desktop) an. Legen Sie **keine Packages** an. Für eine korrekt gelöste Aufgabe gibt es einen Punkt. Halten Sie sich genau an die Vorgaben, gehen Sie davon aus, dass die Programme automatisch getestet werden. Halten Sie sich an die Beispielausgaben, geben Sie Ihren Namen sowie die Aufgabe wie in den Beispielen mit aus.

Zum Bestehen sind **zwei der drei möglichen Punkte nötig**.

Aufgabe 1

In dieser Aufgabe sollen Sie eine generische Klasse DoublyLL schreiben, die eine rudimentäre doppelt verkettete Liste realisiert (Doubly Linked List). Analog zur einfach verketteten Liste, die in der Vorlesung vorgestellt wurde, ist in einem Knoten nicht nur der Nachfolger, sondern auch der Vorgänger gespeichert, s. Illustration:



Ihre Klasse DoublyLL<T> soll nur die Methoden add(T item) und out() für die Ausgabe der Liste von Anfang an haben, sowie backwardout() für die Ausgabe der Liste vom Ende her.

Um die Doubly Linked List aufzubauen, benötigen Sie eine Klasse für einen Knoten LLNode<T>.

Benutzen Sie Ihre Klasse in einer Applikation namens App. Erzeugen Sie 8 Zufallszahlen (Integer) und füllen Sie so eine DoublyLL<Integer> Liste. Nach jeder add() Operation geben Sie die Liste aus. Am Ende geben Sie die Liste einmal rückwärts aus mittels backwardout().

Ihr Programm muss zum Bestehen die obigen Klassen und Funktionalitäten enthalten und beim Start folgende Ausgabe (mit anderen Zahlen) liefern:

Aufgabe 1 von Maxi Musterperson, Start:

12

12 654

12 654 34

12 654 34 56

12 654 34 56 78

12 654 34 56 78 111

12 654 34 56 78 111 52

12 654 34 56 78 111 52 33

Backward:

33 52 111 78 56 34 654 12

Aufgabe 1, Ende.

(1 Punkt)

Aufgabe 2

In dieser Aufgabe sollen Sie ein Programm schreiben, das eine Folge von eingegebenen Integerzahlen analysiert. Die Zahlen sollen durch Leerzeichen getrennt in einer Zeile eingegeben werden. Für jede Zahl in der Folge soll die Quersumme (= Summe der Ziffern) berechnet und ausgegeben werden. Schreiben Sie dazu eine Klasse `NumberAnalyser` mit einer statischen Methode `Integer analyse( Integer i)`, die die Zahl übergeben bekommt, die Quersumme berechnet und zurückgibt.

Benutzen Sie Ihre Klasse in einer Applikation namens `App`. Dort sollen eine Texteingabe per Tastatur entgegengenommen werden, die mehrere durch Leerzeichen getrennte Integerzahlen enthalten kann. Geben Sie anschliessend jede Zahl und die zugehörige Quersumme unter Benutzung der Methode `NumberAnalyser.analyse()` aus.

Ihr Programm muss zum Bestehen die obigen Klassen und Funktionalitäten enthalten und beim Start folgende Ausgabe (auch mit anderen Eingaben) liefern:

Aufgabe 2 von Maxi Musterperson, Start

Eingabe per Tastatur:

12 654 34 56

Ausgabe:

12 hat Quersumme 3

654 hat Quersumme 15

34 hat Quersumme 7

56 hat Quersumme 11

Aufgabe 2, Ende.

(1 Punkt)

Aufgabe 3

Schreiben Sie eine Klasse Haus mit Attribut Hoehe, die Comparable (anhand der Hoehe) implementiert, als statische innere Klasse der Klasse App. Haus soll toString() überschreiben.

Erstellen Sie dann in main() eine List<Haus>, füllen die Liste mit Häusern zufälliger Höhe und geben Sie die Liste aus.

Sortieren Sie die Liste mittels sort() von Collections und geben Sie anschliessend die sortierte Liste aus.

Ihr Programm muss zum Bestehen die obigen Klassen und Funktionalitäten enthalten und beim Start folgende Ausgabe (auch mit anderen Eingaben) liefern:

Aufgabe 3 von Maxi Musterperson, Ende.

unsortiert: [Hoehe: 3, Hoehe: 7, Hoehe: 2, Hoehe: 9, Hoehe: 1]

sortiert: [Hoehe: 1, Hoehe: 2, Hoehe: 3, Hoehe: 7, Hoehe: 9]

Aufgabe 3, Ende.

(1 Punkt)